Translating UML into Code

Software Design & Programming

Object-Oriented Programming II

Joseph DiBiasi

University of Denver College of Professional Studies

4/5/2026

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, MS

Dean: Bobbie Kite, MLS

Introduction

The parking system work that was done this week is not a brand new effort, but a direct continuation from the previous work done to design a parking system for the university. This means that many of the components are not new; previous work focused on laying down a strong backend foundation that relied on a central parking office class to handle most of the logic. That structure is still the foundation; this week we have focused on adding an additional service layer to the backend to facilitate the execution of specific commands. We anticipate these commands being sent from the user interface with a specific set of parameters once the system is fully integrated.

Requirements

The requirements for the development of this service layer are rooted in a loosely coupled design to allow our code flexibility as our application grows. This loosely coupled design also allows for greater testability and easier maintenance once the code itself is active. This is important for our application as we only have the resources to build it piece by piece and the requirements may change as the application development progresses. Current requirements for the new implementation of this code are rooted in our UML design diagram for the parking service. This UML diagram was used to structure the classes as well as model the intended workflow between them. Once design of the new code is implemented, additional backend unit

tests should be created to achieve full coverage of the application.

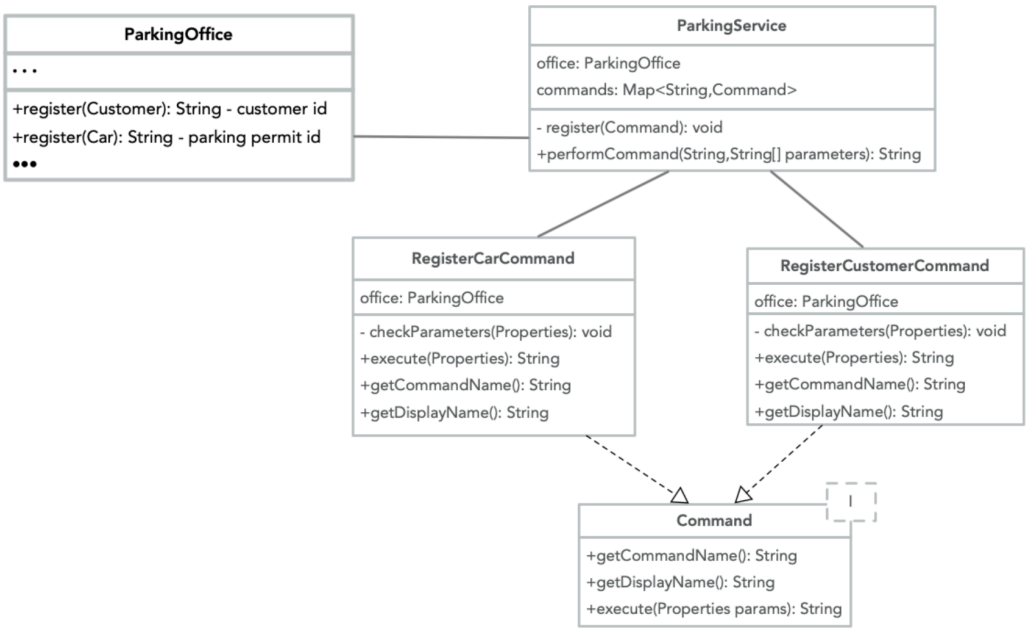

Figure 1. UML Diagram used as Basis for Code Implementation

High-Level Design

The high-level design of the new parking service layer is well mapped out by the UML Diagram. The ParkingService class stands ready to receive input from the pending user interface and use that information to correctly route registration commands to the appropriate command classes. These command classes implement a Command interface to enforce a contract that the behaviors of all command classes will be consistent. These command classes will validate the parameters passed in to ensure they have the needed information before invoking the appropriate registration methods within the ParkingOffice class. Once the ParkingService, RegisterCarCommand, and RegisterCustomerCommand classes were created, test classes were

then created from each and unit tests added within those test classes to achieve full coverage. No test class was created for the interface, Command.

Implementation

Implementation started off by following the UML diagram to create bare bones for the service class, command classes, and command interface. Once these initial bones had been laid down, we looked at the initial desired workflow. This boiled down to wanting a flexible registration method since there were two different methods, one for registering cars and another for customers. By having the ParkingService layer loosely coupled from the actual logic, we can have it adapt to our growing needs as we develop them. Commands can be registered with the ParkingService and once these commands have been registered it is able to execute them at will. It does this by looking at the passed in command name to retrieve the appropriate command. Once this has been done, the retrieved command class takes over and is used to both validate that all required parameters are present as well as execute the actual command logic.

The ParkingService class can be confident of the behavior of these command classes thanks to their implementation of the Command interface. Currently, there are three main behaviors that must be implemented by all classes implementing the Command interface. The getCommandName method is there to ensure the ParkingService is routing to the correct class while getDisplayName provides a user interface friendly version of that command to be used by the front end. The actual execution of the command is handled by an execute method. This method takes the given parameters from the String array initially passed into the ParkingService

and uses the ParkingOffice to execute the actual registration logic needed. The validation of these parameters is not enforced by the interface contract, but individually within each command class. This pattern was directly implemented from the UML diagram, however since parameter validation is currently living in the ParkingService itself, this may be redesigned at a later date to be a more concrete requirement of the command classes.

Once the implementation of the ParkingService and associated Command classes were complete, unit testing was designed to cover all eventualities. In most cases, this was a straightforward continuation of the logic, but the previously alluded to incremental development of the university's parking system did mean that some existing classes were modified to better suit proper testing and development for the application. This was most notable in the register car method which was previously customer focused on the return. This was changed to return a car and better matched the pattern found in the other register method in that the object that was created was also returned. Once these hurdles were overcome, we were able to achieve 100% coverage of the classes found in main. Eclipse tools were used to validate the JUnit testing and coverage as this provided a visual aid of what test cases were covering and what potential edge cases were being missed.

```
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.043 s - in models.TransactionManagerTest
[INFO] Running services.ParkingServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 s - in services.ParkingServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 139, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.10:report (report) @ parking-system ---
[INFO] Loading execution data file C:\Users\campb\Workspace\ICT\ICT_4315\parking_system\target\jacoco.exec
[INFO] Analyzed bundle 'parking-system' with 13 classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.641 s
[INFO] Finished at: 2026-04-05T16:16:15-06:00
[INFO] ------------------------------------------------------------------------
```

Figure 2. Successful Compliation of Code and Test Execution

Summary

While the university parking system continues to grow in both features and complexity, the current application state is still one that needs further development. It is challenging to design a system based on the current requirements when you know those requirements are only going to be a piece of the final puzzle. This can lead to development over-engineering and design choices that are confusing in hindsight, but I have come to believe that is simply the cost of development. It does not matter if it is a professional or academic environment, you should always design an application to best solve the problem as defined by your requirements. Design documents such as UML's are a great tool to aid in this but even a perfect design document will not guarantee a system can be engineered perfectly because in the real world requirements, as well as the tools to meet those requirements, are always changing. We cannot be afraid to change our past work as a result. The world of software design and development is a fast paced one; the best way to keep up is by building loosely coupled applications that meet current needs while remaining flexible enough to evolve over time.